ALGORITHMIQUE EN BASH · JOUR 2 SUR 5 · MARDI 20 OCTOBRE

Mardi · Fonctions, chaînes, fichiers système

1. Fonctions

Une fonction regroupe des instructions sous un nom. À l'intérieur, ses arguments s'appellent `$1`, `$2`… exactement comme pour un script. Le mot `local` garde une variable à l'intérieur de la fonction.

Une fonction bash ne « renvoie » pas de valeur au sens où vous l'entendez ailleurs. Deux techniques :

- elle **affiche** le résultat, et on le récupère avec `$( )` ;
- elle répond oui ou non avec son **code de retour**, qu'on teste directement avec `if`. Vous connaissez le principe depuis la formation Linux : 0 veut dire « succès », donc vrai.

mardi_fonctions.sh

```
#!/usr/bin/env bash
saluer() {
  echo "Salut $1 !"
}

carre() {
  local x=$1
  echo $(( x * x ))          # on affiche le résultat...
}

est_pair() {
  (( $1 % 2 == 0 ))          # ...ou on renvoie vrai/faux
}

saluer "Léa"
c=$(carre 7)                 # ...et on le capture ici
echo "7 au carré : $c"
if est_pair 10; then echo "10 est pair"; fi
if ! est_pair 7; then echo "7 est impair"; fi
```

Sortie

```
Salut Léa !
7 au carré : 49
10 est pair
7 est impair
```

Attention, `$( )` lance une copie du script. Ce que la fonction modifie dans cette copie disparaît ensuite. On s'en souviendra vendredi.

mardi_copie.sh

```
#!/usr/bin/env bash
compteur=0
incremente() { compteur=$(( compteur + 1 )); }

incremente
incremente
echo "Après deux appels : $compteur"

copie=$(incremente; echo "$compteur")
echo "Dans la copie : $copie"
echo "Et ici : $compteur"
```

Sortie

```
Après deux appels : 2
Dans la copie : 3
Et ici : 2
```

2. Tous les arguments d'un coup

`$#` donne le nombre d'arguments, `"$@"` leur liste complète. `shift` retire le premier et décale les autres.

mardi_args.sh

```bash
#!/usr/bin/env bash
echo "Nombre d'arguments : $#"
echo "Premier : $1"
for arg in "$@"; do
  echo "- $arg"
done
```

Exemple d'exécution

```
$ ./mardi_args.sh pomme poire kiwi
Nombre d'arguments : 3
Premier : pomme
- pomme
- poire
- kiwi
```

3. Recherche linéaire

Pour passer un tableau à une fonction, on lui donne tous ses éléments avec `"${t[@]}"`, et la fonction les récupère dans `"$@"` :

mardi_recherche.sh

```bash
#!/usr/bin/env bash
# Affiche l'indice de la première occurrence, ou -1 si absent
chercher() {
  local cible=$1
  shift                       # il ne reste que les valeurs du tableau
  local t=("$@") i
  for i in "${!t[@]}"; do
    if (( t[i] == cible )); then
      echo "$i"
      return
    fi
  done
  echo -1
}

codes=(49000 44000 75001 49100 35000)
echo "44000 -> $(chercher 44000 "${codes[@]}")"
echo "13000 -> $(chercher 13000 "${codes[@]}")"
```

Sortie

```
44000 -> 1
13000 -> -1
```

C'est la **recherche linéaire** : dans le pire des cas, on regarde chaque case une fois.

4. Chaînes de caractères

Syntaxe	Résultat pour `s="algorithme"`
`${#s}`	10 (la longueur)
`${s:2:1}`	`g` (1 caractère à partir de l'indice 2)
`${s:0:4}`	`algo`
`${s:4}`	`rithme` (de l'indice 4 jusqu'à la fin)
`s+="s"`	ajoute un « s » à la fin

Parcourir une chaîne lettre par lettre, c'est une boucle sur les indices :

mardi_voyelles.sh

```
#!/usr/bin/env bash
mot="algorithme"
voyelles=0
for (( i = 0; i < ${#mot}; i++ )); do
  c=${mot:i:1}
  case $c in
    [aeiouy]) (( voyelles++ )) ;;
  esac
done
echo "$voyelles voyelles dans $mot"
```

Sortie

```
4 voyelles dans algorithme
```

Évitez les accents dans les chaînes que vous découpez : selon la configuration de la machine, « é » peut compter pour deux caractères.

5. Lire un fichier ligne par ligne

Les algorithmes de la semaine travaillent sur des listes de valeurs. Sur un serveur, ces listes sont dans des fichiers : `/etc/passwd`, les journaux, la sortie d'une commande. La boucle `while read` les lit une ligne à la fois. Avec `IFS=:`, chaque ligne est découpée sur les `:` et chaque morceau va dans une variable. Le `_` sert de poubelle pour les champs qui ne nous intéressent pas.

Pour que tout le monde obtienne les mêmes résultats, on travaille sur une copie réduite de `/etc/passwd`. Créez ce fichier à côté de vos scripts :

passwd_exemple.txt

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
sshd:x:110:65534::/run/sshd:/usr/sbin/nologin
alice:x:1000:1000:Alice:/home/alice:/bin/bash
bob:x:1001:1001::/home/bob:/bin/zsh
chloe:x:1002:1002::/home/chloe:/bin/bash
david:x:1003:1003::/home/david:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
```

mardi_lecture.sh

```bash
#!/usr/bin/env bash
fichier=${1:-/etc/passwd}
lignes=0
humains=0
while IFS=: read -r login _ uid _ _ home shell; do
  (( lignes++ ))
  if (( uid >= 1000 && uid < 65534 )); then
    echo "$login ($uid) : $home, $shell"
    (( humains++ ))
  fi
done < "$fichier"
echo "$humains comptes utilisateurs sur $lignes lignes"
```

Exemple d'exécution

```
$ ./mardi_lecture.sh passwd_exemple.txt
alice (1000) : /home/alice, /bin/bash
bob (1001) : /home/bob, /bin/zsh
chloe (1002) : /home/chloe, /bin/bash
david (1003) : /home/david, /bin/sh
4 comptes utilisateurs sur 11 lignes
```

Sans argument, le script lit le vrai `/etc/passwd` de votre VM. Essayez.

Exercices du mardi

M1. max2 et max3 NIVEAU 1

Écrire une fonction `max2` qui affiche le plus grand de deux nombres, puis une fonction `max3` qui n'utilise **que** `max2`, sans aucun `if`.

Exemple d'exécution

```
$ ./max.sh
max2 4 9     -> 9
max3 4 9 2   -> 9
max3 12 9 2  -> 12
```

M2. Moyenne NIVEAU 1

Les notes arrivent en arguments. Afficher la somme, la moyenne entière, puis la moyenne avec deux décimales. Bash ne sait pas faire de décimales, mais `awk` oui : `awk -v s=71 -v c=6 'BEGIN { printf "%.2f", s / c }'`.

Exemple d'exécution

```
$ ./moyenne.sh 12 8 15 10 17 9
Somme : 71
Moyenne entière : 11
Moyenne exacte : 11.83
```

M3. Inverser une chaîne NIVEAU 2

Fonction `inverser` qui affiche la chaîne reçue à l'envers. La commande `rev` existe, elle est interdite.

Exemple d'exécution

```
$ ./inverser.sh bonjour
ruojnob
$ ./inverser.sh algorithme
emhtirogla
```

M4. Palindrome NIVEAU 2

Fonction `est_palindrome` qui répond par son code de retour, sans fabriquer la chaîne inversée : comparez la première lettre à la dernière, la deuxième à l'avant-dernière, etc. Bonus : ignorer les espaces et les majuscules (`tr 'A-Z' 'a-z'` passe en minuscules, `tr -d ' '` retire les espaces).

Exemple d'exécution

```
$ ./palindrome.sh kayak bash 'Esope reste ici et se repose'
« kayak » est un palindrome
« bash » n'est pas un palindrome
« Esope reste ici et se repose » est un palindrome
```

M5. PGCD d'Euclide NIVEAU 2

Le plus grand diviseur commun de `a` et `b` : tant que `b` ne vaut pas 0, on remplace `(a, b)` par `(b, a % b)`. Quand `b` vaut 0, la réponse est `a`. Déroulez l'algorithme à la main sur 84 et 36 avant de le coder.

Exemple d'exécution

```
$ ./pgcd.sh 84 36
pgcd(84, 36) = 12
$ ./pgcd.sh 1071 462
pgcd(1071, 462) = 21
$ ./pgcd.sh 17 5
pgcd(17, 5) = 1
```

M6. Décimal vers binaire NIVEAU 2

On divise par 2 tant que le nombre est positif, et on note les restes. Lus dans l'ordre inverse, ils donnent l'écriture binaire. Vérifiez vos résultats avec `echo $(( 2#1101 ))`, qui convertit du binaire vers le décimal.

Exemple d'exécution

```
$ ./binaire.sh 13
13 en binaire : 1101
$ ./binaire.sh 255
255 en binaire : 11111111
$ ./binaire.sh 1024
1024 en binaire : 10000000000
```

M7. Supprimer les doublons NIVEAU 3

Afficher la liste sans doublons, dans l'ordre de première apparition.

Exemple d'exécution

```
$ ./doublons.sh 4 2 4 7 2 9 4
4 2 7 9
```

M8. Qui utilise quel shell ? NIVEAU 2

Pour chaque shell présent dans le fichier, afficher le nombre de comptes qui l'utilisent, dans l'ordre où les shells apparaissent. En formation Linux, vous auriez tapé `cut -d: -f7 fichier | sort | uniq -c`. Ici, ni `cut`, ni `sort`, ni `uniq` : une boucle `while read` et un tableau associatif suffisent. Testez sur `passwd_exemple.txt`, puis sur le vrai `/etc/passwd`.

Exemple d'exécution

```
$ ./shells.sh passwd_exemple.txt
  3  /bin/bash
  5  /usr/sbin/nologin
  1  /bin/sync
  1  /bin/zsh
  1  /bin/sh
```

M9. Les trois plus gros fichiers NIVEAU 3

Afficher les trois plus gros fichiers d'un dossier (par défaut `/var/log`), en un seul parcours et sans `sort`. Les sous-dossiers sont ignorés. `stat -c %s fichier` donne la taille en octets. C'est l'exercice du deuxième plus grand (L10), avec un podium à trois places.

Exemple d'exécution

```
$ ./top3.sh logs_exemple
  152004  logs_exemple/syslog
   73550  logs_exemple/dpkg.log
   48213  logs_exemple/auth.log
```

Sur `/var/log`, certains fichiers ne sont lisibles qu'avec `sudo`. Votre script n'a pas besoin de les lire : `stat` donne leur taille quand même.

Mémo bash

Besoin	Syntaxe
Créer une variable	`nom="Robin"` (sans espaces)
Lire au clavier	`read -p "Question : " rep`
Arguments du script	`$1`, `$2`, `$#` (nombre), `"$@"` (tous)
Valeur par défaut	`n=${1:-100}`
Calcul	`x=$(( a * 2 + 1 ))`, `(( x++ ))`
Nombre aléatoire de 1 à 100	`$(( RANDOM % 100 + 1 ))`
Décimales	`awk 'BEGIN { printf "%.2f", 7 / 3 }'`
Comparer des nombres	`(( a < b ))`, `(( a == b ))` (plutôt que `-lt`, `-eq`)
Comparer des chaînes	`[[ $s == "oui" ]]`, `[[ -z $s ]]`
Condition	`if ...; then ...; elif ...; then ...; else ...; fi`
Boucle sur une plage	`for i in {1..10}; do ...; done`
Boucle avec variable	`for (( i = 0; i < n; i++ )); do ...; done`
Boucle tant que	`while (( x > 0 )); do ...; done`
Fonction	`f() { local a=$1; echo "$a"; }`
Récupérer le résultat	`r=$(f 42)`
Tableau	`t=(3 1 2)`, `t+=(5)`, `${t[0]}`
Tableau associatif	`declare -A p; p[ssh]=22; ${p[ssh]}`
Lire un fichier	`while IFS=: read -r a b _; do ...; done < fichier`
Taille, éléments, indices	`${#t[@]}`, `"${t[@]}"`, `"${!t[@]}"`
Longueur d'une chaîne	`${#s}`
Sous-chaîne	`${s:debut:longueur}`, `${s: -1}`
Contient	`[[ $s == *"$c"* ]]`
Déboguer	`bash -x script.sh`
Mesurer le temps	`time ./script.sh`